

Sadržaj

1. Analiza izvršne specifikacije	2
1.1. Matematički aparat.....	2
1.1.1. Matematička definicija NCC-a.....	2
1.1.2. Implementacioni aspekti.....	3
1.1.3. Algoritamska optimizacija za Hardver	3
1.1.4. Prag detekcije (Threshold).....	3
1.2. Vremensko profilisanje	4
2. Bitska analiza	6
3. Particionisanje sistema.....	7
4. Moduli virtuelne platforme	9
4.1. Interconnect (Sabirnica)	9
4.2. DMA (Direct Memory Access) modul.....	9
4.3. NCC (Normalized Cross-Correlation) IP modul.....	10
4.4. BRAM modul	11
4.5. DDR memorijski modul	11
4.6. Instanciranje modula	12
5. Hardversko-softverski interfejs.....	13
6. Registarska mapa	14
6.1. Registri NCC modula	14
6.2. Registri DMA modula.....	14
7. Modelovanje vremena kašnjenja.....	15
7.1. HLS latencija NCC IP-ja.....	15
7.2. BRAM kašnjenje	16
7.3. DMA i DDR kašnjenje	16
8. Optimizacije i paralelizacija	17
8.1. Analiza vremena izvršavanja (pre optimizacija).....	17
8.2. HLS resursna analiza i broj NCC blokova	17
8.3. Paralelizacija (2 NCC bloka).....	18
8.4. Optimizacije	19
8.4.1. Provera praznog polja.....	19
8.4.2. Keširanje učitavanja segmenta	19
8.4.3. Pred-klasifikacija boje figura.....	19
8.4.4. Coarse-to-fine (downsampling 2×).....	20
8.5. Analiza vremena izvršavanja nakon optimizacija.....	21
8.5.1. Model latencije zavisao od veličine posla	21
8.5.2. Dekompozicija vremena po popunjenom polju.....	21
8.5.3. Pregled rezultata	22

1. Analiza izvršne specifikacije

Host platforma izvršne specifikacije je računar sa Linux operativnim sistemom. Njegove mogućnosti ne mogu da se mere sa mogućnostima target platforme Zybo Z7 i ZYNQ 7010 sistema na čipu. Zbog toga trenutne rezultate analize treba tumačiti kao okvirne linije vodilje ka procesu particionisanja hardvera i softvera.

Izvršna specifikacija je softverski model napisan u MATLAB okruženju (FEN_notacija.m i fengenerator.m) koji vrši učitavanje slike šahovske table (720×720 piksela), njenu segmentaciju na 64 polja dimenzija 90×90 piksela, i prepoznavanje šahovskih figura upotrebom normalizovane unakrsne korelacije (engl. *Normalized Cross-Correlation* - NCC). Nakon prepoznavanja, model generiše standardnu Forsyth-Edwards (FEN) notaciju.

1.1. Matematički aparat

Da bismo precizno definisali proces prepoznavanja figura, uvodimo matematički aparat koji stoji iza normalizovane unakrsne korelacije (NCC). Ova metoda je robusnija od obične korelacije jer je imuna na promene u osvetljenju (amplitudi signala) između slike i templejta.

1.1.1. Matematička definicija NCC-a

Za svaki segment šahovske table f dimenzija 90×90 i templejt figure t dimenzija 30×30 , koeficijent korelacije γ u tački (u, v) izračunava se prema sledećoj formuli:

$$\gamma(u, v) = \frac{\sum_{x,y} [f(x, y) - \bar{f}_{u,v}] [t(x - u, y - v) - \bar{t}]}{\sqrt{\sum_{x,y} [f(x, y) - \bar{f}_{u,v}]^2 \sum_{x,y} [t(x - u, y - v) - \bar{t}]^2}}$$

Gde su:

- $f(x, y)$ – vrednost intenziteta piksela na segmentu slike;
- $\bar{f}_{u,v}$ – srednja vrednost piksela na delu slike koji se trenutno preklapa sa templejtom;
- $t(x - u, y - v)$ – vrednost piksela templejta figure;
 $\bar{t}$ – srednja vrednost piksela celog templejta (unapred izračunata vrednost).

1.1.2. Implementacioni aspekti

U MATLAB-u, funkcija `normxcorr2` koristi brzu Furijeovu transformaciju (FFT) za računanje brojioca, dok imenilac računa pomoću pokretnih suma (engl. *running sums*). Matematički profil uskog grla koji smo identifikovali u profilisanju može se dekomponovati na tri ključne operacije:

1. **Lokalno sumiranje:** Izračunavanje $\bar{f}_{u,v}$ za svaki prozor, što zahteva $O(m \times n)$ operacija.
2. **Korelacija u brojiocu:** Direktna konvolucija ili FFT, gde je složenost dominantna.
3. **Normalizacija:** Operacija deljenja i korenovanja po svakom pikselu rezultatne mape.

1.1.3. Algoritamska optimizacija za Hardver

S obzirom na to da su operacije korenovanja i deljenja izuzetno skupe u hardveru (FPGA), u našem SystemC modelu vršimo optimizaciju tako što poredimo kvadriranu vrednost korelacije sa kvadriranim pragom detekcije:

$$\gamma^2(u, v) > threshold^2$$

Ovim se eliminiše potreba za izračunavanjem kvadratnog korena u svakom ciklusu NCC akceleratora, čime se značajno štedi na resursima Zybo Z7 ploče i smanjuje vreme obrade jednog polja.

1.1.4. Prag detekcije (Threshold)

U našem sistemu, vrednost γ se kreće u opsegu $[-1, 1]$.

- Vrednost **1** označava savršeno poklapanje.
- Vrednost **0** označava potpuno odsustvo korelacije.
- Vrednost **-1** označava inverzno poklapanje.

Eksperimentalno je utvrđeno da je za prepoznavanje figura na *board2.png* optimalan prag $\gamma = 0.75$. Sve vrednosti ispod ovog praga u `result_t` (fixed-point tipu) NCC modul interpretira kao prazno polje (oznaka 0 u matrici za `fengenerator.m`).

1.2. Vremensko profilisanje

Nad softverskim modelom pokrenut je MATLAB profajler kako bi se dobio uvid u ukupan broj poziva funkcija i vreme njihovog izvršavanja, kao i funkcije u modelu koje zauzimaju najveći udeo u procesorskom vremenu (engl. *hotspots*).

Funkcija	Ukupno vreme [s]	Sopstveno vreme [s]	Broj poziva
normxcorr2	0.730	0.154	416
normxcorr2>local_sum	0.278	0.114	832
normxcorr2>xcorr2_fast	0.238	0.054	416
normxcorr2>freqxcorr	0.179	0.018	288

Tabela 1. Rezultati vremenskog profilisanja softverskog modela

Profile Summary
Generated 02-Apr-2026 14:04:40 using performance time.

Function Name	Calls	Total Time	Self Time*	Total Time Plot (dark band = self time)
normxcorr2	416	0.730 s	0.154 s	
normxcorr2>local_sum	832	0.278 s	0.114 s	
normxcorr2>xcorr2_fast	416	0.238 s	0.054 s	
normxcorr2>freqxcorr	288	0.179 s	0.018 s	
imshow	2	0.166 s	0.063 s	
padarray	832	0.164 s	0.016 s	
fft2	576	0.097 s	0.097 s	
images\private\padarray_algo	832	0.066 s	0.005 s	
images\private\padarray_algo>ConstantPad	832	0.061 s	0.033 s	
stringToChar	833	0.061 s	0.047 s	
fft2	288	0.061 s	0.061 s	
normxcorr2>ParseInputs	416	0.049 s	0.022 s	
initSize	1	0.041 s	0.004 s	
imread	13	0.034 s	0.009 s	
movegui	1	0.029 s	0.001 s	
images\private\mkconstarray	832	0.028 s	0.028 s	
legacyMoveGUI	1	0.026 s	0.015 s	
std	832	0.025 s	0.006 s	
padarray>ParseInputs	832	0.024 s	0.024 s	
var	832	0.019 s	0.016 s	
imageDisplayParseInputs	2	0.017 s	0.001 s	
imread>call_format_specific_reader	13	0.017 s	0.002 s	
basicImageDisplay	2	0.017 s	0.006 s	
normxcorr2>checkHFlat	416	0.017 s	0.003 s	
imagesc\private\readpng	13	0.015 s	0.001 s	
imageDisplayValidateParams	2	0.014 s	0.005 s	
stringToChar>@(x)convertToChar(x)	1666	0.013 s	0.009 s	
imagesc\private\readpngutil	13	0.013 s	0.004 s	

Slika 2. Izlaz MATLAB profajlera (Profile Summary) sa rang-listom funkcija po vremenu

Kao što se može primetiti iz Tabele 1, usko grlo sistema je funkcija `normxcorr2` (normalizovana unakrsna korelacija) i njene potfunkcije za lokalno sumiranje i brzu korelaciju. S obzirom na to da se korelacija računa za svaki od 64 segmenta (90×90 piksela) sa više različitih templejta figura, ova operacija dominantno troši računarske resurse. Zaključuje se da je glavni kandidat za hardversku implementaciju i ubrzanje upravo IP jezgro za računanje NCC-a.

2. Bitska analiza

Hardverska implementacija na FPGA zahteva prelazak sa aritmetike u pokretnom zarezu (floating-point, koja je podrazumevana u MATLAB-u) na aritmetiku u fiksnom zarezu (fixed-point). Time se vrši ušteda logičkih resursa (DSP slajseva i LUT-ova) uz kontrolisan gubitak preciznosti. Ključna odluka u našem modelu je zaokruživanje srednjih vrednosti (f_{bar} i template_mean) na ceo broj: time razlike piksel – sredina postaju egzaktni 9-bitni celi brojevi, pa je čitav datapath bit-egzaktan i nema akumulacije greške zaokruživanja kroz sumiranje. Sve međuvrednosti su zato celobrojni tipovi (sc_int / sc_uint), dok je jedini pravi fixed-point tip normalizovani rezultat NCC^2 , koji se čuva u Q1.31 formatu ($\text{sc_ufixed}<32,1>$).

Promenljiva/tip u modelu	Tip podataka(SystemC)	Ukupan broj bita	Broj bita za celobrojni deo
$f(\text{image})$, $t(\text{template})$ (vrednost piksela)	$\text{sc_uint}<8>$	8	8
sum_f (suma piksela prozora)	$\text{sc_uint}<18>$	18	18
f_{bar} , template_mean (zaokr. sredina)	$\text{sc_uint}<8>$	8	8
diff_f , diff_t (piksel - sredina)	$\text{sc_uint}<9>$	9	9
sum_num (suma proizvoda)	$\text{sc_uint}<27>$	27	27
sum_den_f , sum_den_t (suma kvadrata)	$\text{sc_uint}<26>$	26	26
num_sq (kvadrat brojioca)	$\text{sc_uint}<52>$	52	52
den_prod (proizvod imenilaca)	$\text{sc_uint}<52>$	52	52
NCC^2 u Q1.31 (norm. rezultat)	$\text{sc_ufixed}<32,1>$	32	1
result_t (tip povratne vrednosti NCC bloka u kodu)	$\text{sc_uint}<32>$	32	32

Tabela 2. Bitske širine promenljivih i tipova NCC modula

3. Partitionisanje sistema

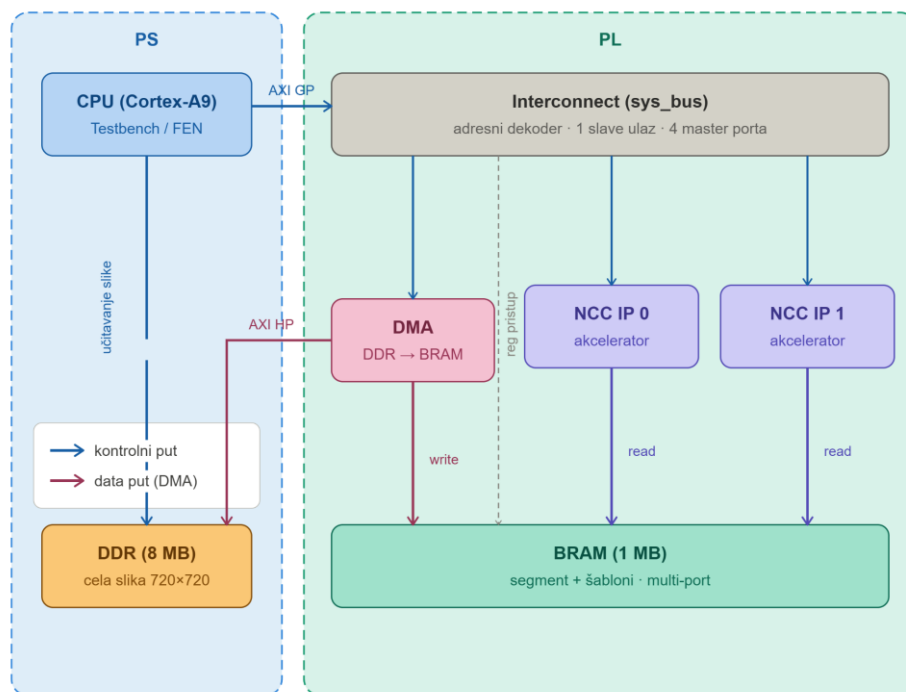
Na osnovu prethodne analize vremenskog profilisanja, utvrđeno je da funkcija `normxcorr2` troši preko 70% procesorskog vremena. Cilj partitionisanja je prebacivanje ove kritične operacije u namenski hardverski akcelerator (NCC IP jezgro) i funkcionalna dekompozicija celokupnog sistema.

Ostatak logike, koji podrazumeva kontrolu toka programa, učitavanje slika, rad sa FEN notacijom i upravljanje periferijama, ostaje na softverskoj strani (CPU). Ovakva podela omogućava paralelno izvršavanje gde procesor priprema sledeći segment slike dok hardver vrši korelaciju na trenutnom.

Virtuelna platforma je razvijena u SystemC jeziku uz korišćenje TLM 2.0 (Transaction Level Modeling) standarda. TLM omogućava visoku brzinu simulacije modelovanjem komunikacije putem transakcija.

Sistem se sastoji od:

- **Inicijatora (Initiator):** Testbench ili procesorski model koji generiše saobraćaj.
- **Ciljeva (Targets):** NCC IP 0, NCC IP 1, DMA, BRAM i DDR koji odgovaraju na zahteve.
- **Sabirnice (Interconnect):** Vršiti ruting transakcija na osnovu adresne mape.



Slika 1. Blok šema virtualne platforme

Ključna karakteristika ovakvog partitionisanja je razdvajanje kontrolnog puta (CPU pristupa periferijama preko interkonekta) od podatkovnog puta (DMA čita iz DDR-a i piše u BRAM, a NCC samostalno čita iz BRAM-a) – podatkovni saobraćaj ne prolazi kroz interkonekt, čime se izbegava usko grlo magistrale. Funkcija `b_transport_bus` u fajlu `vp.cpp` prikazuje dekodiranje adresa za kontrolni put:

```
void vp::b_transport_bus(tlm_generic_payload& trans, sc_time& delay) {
    uint64_t addr = trans.get_address();
    if (addr < ADDR_BRAM)
        i_ddr->b_transport(trans, delay);      // PS: DDR
    else
        s_bus_int->b_transport(trans, delay); // PL: BRAM/NCC/DMA
}
```


4. Moduli virtuelne platforme

4.1. Interconnect (Sabirnica)

Sabirnica predstavlja centralno komunikaciono čvorište sistema i modelovan je korišćenjem TLM 2.0 standarda radi postizanja visoke brzine simulacije. Njena primarna uloga je dekodiranje adresa koje procesor generiše i usmeravanje transakcija ka ostalim modulima kao što su DMA, NCC ili memorije. Implementiran je pomoću `simple_target_socket` i `simple_initiator_socket` utičnica koje omogućavaju transparentan prolaz transakcija uz minimalno kašnjenje modela.

Modul koristi statičku adresnu mapu definisanu u zaglavlju sistema (`common.hpp`) kako bi odredio koji modul treba da odgovori na specifičan zahtev za čitanje ili upis. U narednom pasusu dat je deo funkcije `b_transport` koji prikazuje način na koji se vrši prepoznavanje i translacija adresa (ovde konkretno za DMA modul):

```
uint64_t addr = trans.get_address();

if (addr >= ADDR_DMA) {
    trans.set_address(addr - ADDR_DMA);
    i_dma->b_transport(trans, offset);
    trans.set_address(addr);
}
```

4.2. DMA (Direct Memory Access) modul

DMA modul omogućava efikasan prenos velikih blokova podataka između DDR i BRAM memorija bez direktnog učešća procesora u samom procesu kopiranja.

Specifičnost ovog modula je podrška za "stride" (korak) parametar, koji omogućava da DMA iz veće slike u DDR-u (720x720) "izreže" i prebaci samo relevantan 90x90 segment slike u lokalni BRAM.

Programiranje DMA modula vrši se upisom izvorne adrese, odredišne adrese i dužine podataka u njegove konfiguracione registre. Jednom pokrenut, DMA preuzima ulogu "Mastera" na sabirnici, inicirajući transakcije čitanja i upisa sve dok se prenos ne završi. Na ovaj način, procesor ostaje slobodan da izvršava druge zadatke, kao što je generisanje FEN notacije za prethodno obrađena polja, čime se postiže preklapanje rada hardvera i softvera.

4.3. NCC (Normalized Cross-Correlation) IP modul

NCC IP modul je namenski hardverski akcelerator dizajniran da preuzme najteži deo proračuna — prepoznavanje šahovskih figura na segmentima slike.

Modul bi trebalo da radi u fiksnom zarezu koristeći `sc_fixed` tipove podataka dobijenih u bitskoj analizi algoritma, ali za potrebe simulacije se koriste standardni tipovi (`double`, `uint32_t`,...). Interno, modul sadrži lokalne bafere u koje smešta piksele segmenta (45x45) i templejta figura kako bi minimizovao pristup eksternoj memoriji tokom izvršavanja dvostrukih petlji korelacije. Logika modula implementira optimizovanu verziju NCC formule gde se izbegava operacija kvadratnog korena poređenjem **kvadriranih vrednosti korelacije** sa unapred definisanim pragom. Kontrola rada se vrši preko statusnih i kontrolnih registara, gde procesor upisuje komandu za start, a zatim čeka "Done" signal ili proverava statusni bit.

Centralna optimizacija algoritma je **Summed-Area Table (SAT)**, poznata i kao integral image. Gradi se jednom po segmentu prolaskom kroz sve piksele slike, a potom omogućava računanje sume piksela bilo kog pravougaonog prozora u $O(1)$ (konstantnom) vremenu korišćenjem samo četiri pristupa memoriji, umesto dvostruke petlje složenosti $O(tmp_w \times tmp_h)$. Ovo je posebno značajno u kontekstu RTL implementacije gde svaki pristup memoriji nosi cenu u taktovima.

Modul je modelovan kao `SC_THREAD` proces koji radi paralelno sa ostatkom sistema. Kada procesor upiše komandu za start u kontrolni registar (`REG_CTRL = 1`), `b_transport` funkcija samo okine `start_ev` događaj i odmah se vrati, ne blokirajući procesor. Sama obrada teče u zasebnoj niti koja po završetku postavlja `hw_status = 1` i emituje `done_ev` signal, na koji testbench čeka kako bi pročitao rezultate.

Učitavanje podataka iz BRAM-a nalazi se na kritičnoj putanji izvršavanja — modul mora da pribavi segment slike i šablon pre nego što može da započne korelaciju, pa se vreme ovog prenosa modeluje i naplaćuje kroz `wait()` mehanizam. Kako bi se izbeglo nepotrebno ponavljanje prenosa, implementiran je `img_dirty` flag: segment slike i njegova Summed-Area Table grade se samo kada procesor upiše novu adresu u `REG_IMG_ADDR`, a ne na svakom od 12 šablona. Šablon se, nasuprot tome, uvek iznova učitava jer se menja pri svakom pokretanju.

4.4. BRAM modul

BRAM modul predstavlja brzu on-chip memoriju FPGA uređaja i u virtuelnoj platformi služi kao primarni radni prostor za NCC akcelerator. Karakteriše ga izuzetno niska latencija (u modelu definisana kao 10 ns po ciklusu), što omogućava NCC jezgru da povlači piksele u svakom ciklusu bez zastoja.

Memorijski prostor od 1MB je dovoljan da se u njega smeste trenutni 45x45 segment slike i set templejta potrebnih za poređenje. BRAM je implementiran kao TLM 2.0 `multi_passthrough_target` koji podržava blokirajuće transporte, simulirajući ponašanje memorije koja prihvata transakcije od više mastera (ovde su to DMA, NCC0, NCC1, Interconnect).

U finalnoj implementaciji na Zybo Z7, ovaj modul se mapira na fizičke BRAM resurse unutar Zynq programabilne logike.

4.5. DDR memorijski modul

DDR modul modeluje eksternu RAM memoriju sistema u kojoj se nalazi kompletna ulazna slika šahovske table i baza svih templejta figura.

Kao i BRAM, i DDR je implementiran kao TLM 2.0 `multi_passthrough_target` koji podržava blokirajuće transporte od dva mastera (CPU i DMA). CPU mu pristupa jednom, na samom početku simulacije, kako bi upisao sadržaj slike iz fajla u memorijski prostor. Nakon toga, DMA preuzima ulogu primarnog korisnika DDR-a — za svako polje šahovske table DMA čita odgovarajući 90×90 segment kroz AXI HP interfejs i prenosi ga u BRAM.

Kapacitet od 8MB je rezervisan za smeštanje sirovih podataka slike učitanih iz datoteke pre početka obrade. DDR služi isključivo kao izvor podataka za DMA prenose, jer bi direktno čitanje piksela od strane NCC modula iz DDR-a rezultovalo neprihvatljivo sporom simulacijom i radom.

U virtuelnoj platformi, ovaj modul je implementiran kao veliki niz bajtova sa pridruženim metodama za rukovanje transakcijama koje poštuju definisane vremenske parametre. U realnom slučaju, DDR čini deo PS memorijskog prostora.

U finalnoj implementaciji na Zybo Z7-10, ovaj modul odgovara fizičkoj DDR3L memoriji od 1 GB koja je deo PS podsistema i dostupna je PL strani kroz AXI HP (High Performance) port Zynq procesora.

4.6. Instanciranje modula

Naredni deo koda prikazuje instanciranje modula i njihovo povezivanje u fajlu `vp.cpp`:

```
bus = new sys_bus("bus");
ncc0 = new NCC_Target("ncc0");
ncc1 = new NCC_Target("ncc1");
bram = new BRAM_Module("bram");
ddr = new DDR_Module("ddr");
dma = new DMA_Module("dma");

s_cpu.register_b_transport(this, &vp::b_transport_bus);
s_bus_int.bind(bus->s_cpu);
i_ddr.bind(ddr->socket);

bus->i_ncc.bind(ncc0->socket);
bus->i_ncc1.bind(ncc1->socket);
bus->i_bram.bind(bram->socket);
bus->i_dma.bind(dma->s_cpu);

dma->i_rd.bind(ddr->socket);
dma->i_wr.bind(bram->socket);

ncc0->i_bram.bind(bram->socket);
ncc1->i_bram.bind(bram->socket);
```

Uočava se da DMA i NCC nisu vezani na interkonekt kao podatkovni masteri: DMA čita direktno iz DDR-a (`i_rd`) i piše u BRAM (`i_wr`), dok NCC samostalno čita iz BRAM-a (`i_bram`). Interkonekt opslužuje isključivo kontrolni put procesora. Ovakav način povezivanja prikazan je i na blok šemi sistema.

5. Hardversko-softverski interfejs

Komunikacija između softvera i hardvera se vrši putem **memorijski mapiranog I/O-a (MMIO)**. CPU pristupa registrima hardverskih modula kao da su obične memorijske lokacije.

Sistem koristi `common.hpp` zaglavlje gde su definisane sve bazne adrese:

```
ADDR_BRAM = 0x40000000
ADDR_NCC  = 0x50000000
ADDR_NCC1 = 0x51000000
ADDR_DMA  = 0x60000000
ADDR_DDR  = 0x00000000
```

Nakon upisa komande u `REG_CTRL`, NCC po završetku emituje Done signal (modelovan kao prekidna linija / `done_ev`) na koji procesor čeka pre čitanja rezultata sa `ADDR_RESULTS`. Statusni registar `REG_STATUS` je dostupan za polling, ali se u simulaciji koristi efikasnija sinhronizacija preko Done događaja. Kada se obrađuju oba NCC bloka CPU čeka da oba signaliziraju Done.

6. Registarska mapa

6.1. Registri NCC modula

Za kontrolu i konfiguraciju NCC hardverskog akceleratora definisana je mapa dinamičkih registara. Hardverskom modulu je potrebno proslediti dimenzije slike, dimenzije templejta, kao i adrese na kojima se nalaze pikseli slike i templejta kako bi se korelacija uspešno izvršila.

Naziv registra / Adresa	Ofset (Hex)	Opis
REG_IMG_W	0x00	Širina ulazne slike / segmenta
REG_IMG_H	0x04	Visina ulazne slike / segmenta
REG_TMP_W	0x08	Širina templejta (šahovske figure)
REG_TMP_H	0x0C	Visina templejta
BRAM_TMP_COARSE	0x10000	Grubi (2x smanjeni) šabloni
BRAM_SEG_FULLL	0x80000	Pun 90x90 segment slike
BRAM_SEG_COARSE	0x90000	Grubi 45x45 segment slike
REG_IMG_ADDR	0x10	BRAM adresa segmenta slike
REG_TMP_ADDR	0x14	BRAM adresa templejta
REG_CTRL	0x30	Kontrolni registar (Start, Reset, Enable)
REG_STATUS	0x34	Statusni registar (Idle, Busy, Done)
ADDR_RESULTS	0x40	Adresa na koju se upisuje rezultat NCC-a

Tabela 3. Bitska analiza memorijske mape NCC akceleratora

6.2. Registri DMA modula

Sljedeća tabela prikazuje registre DMA modula.

Adresa (Offset)	Simboličko ime	Opis
0x00	DMA_SRC_ADDR	Izvorna adresa u DDR-u
0x08	DMA_DST_ADDR	Ciljna adresa u BRAM-u
0x10	DMA_WIDTH	Broj bajtova po redu
0x14	DMA_HEIGHT	Broj redova
0x18	DMA_SRC_STRIDE	Korak u izvornoj memoriji
0x1C	DMA_DST_STRIDE	Korak u odredišnoj memoriji
0x20	DMA_START	Upisom jedinice kreće prenos

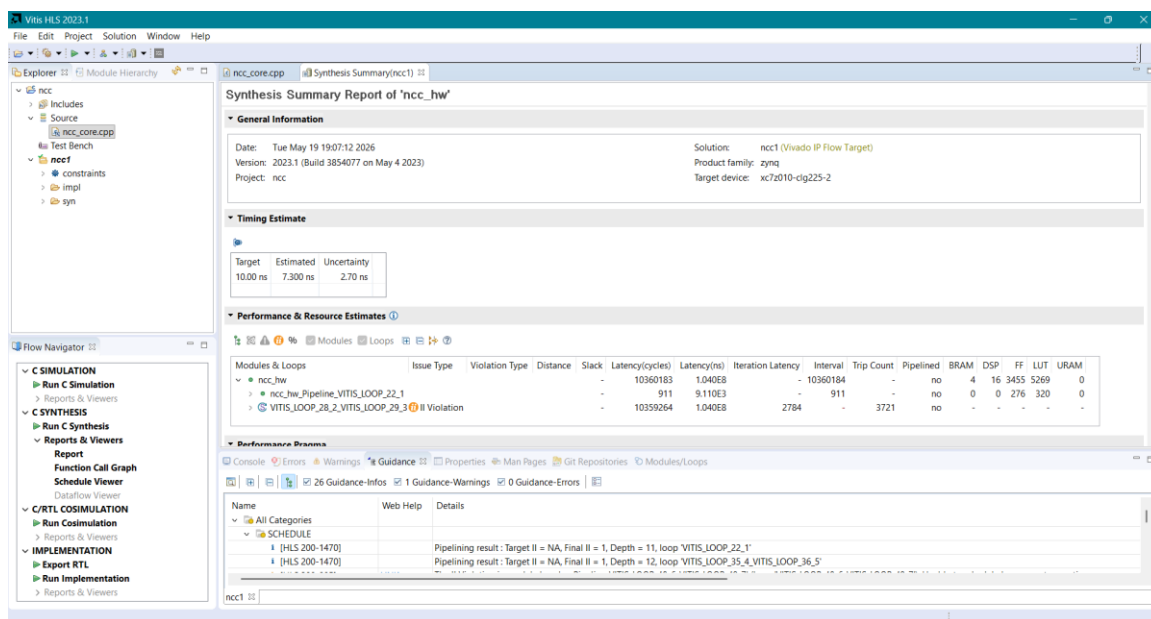
Tabela 4. Konfiguracija DMA modula

7. Modelovanje vremena kašnjenja

7.1. HLS latencija NCC IP-ja

Funkcionalnost **NCC** jezgra opisana je u jeziku C++ i sintetizovana alatom Vitis HLS 2023.1 za ciljni uređaj xc7z010-clg225-2 (Zynq-7010). Procena perioda takta iznosi 7.3 ns pri zadatom cilju od 10 ns, što ostavlja pozitivnu vremensku rezervu. Latencija jednog pokretanja jezgra je **10.360.183** ciklusa, a utrošeni resursi su 4 BRAM bloka, 16 DSP jedinica, 3455 flip-flopora i 5269 LUT-ova.

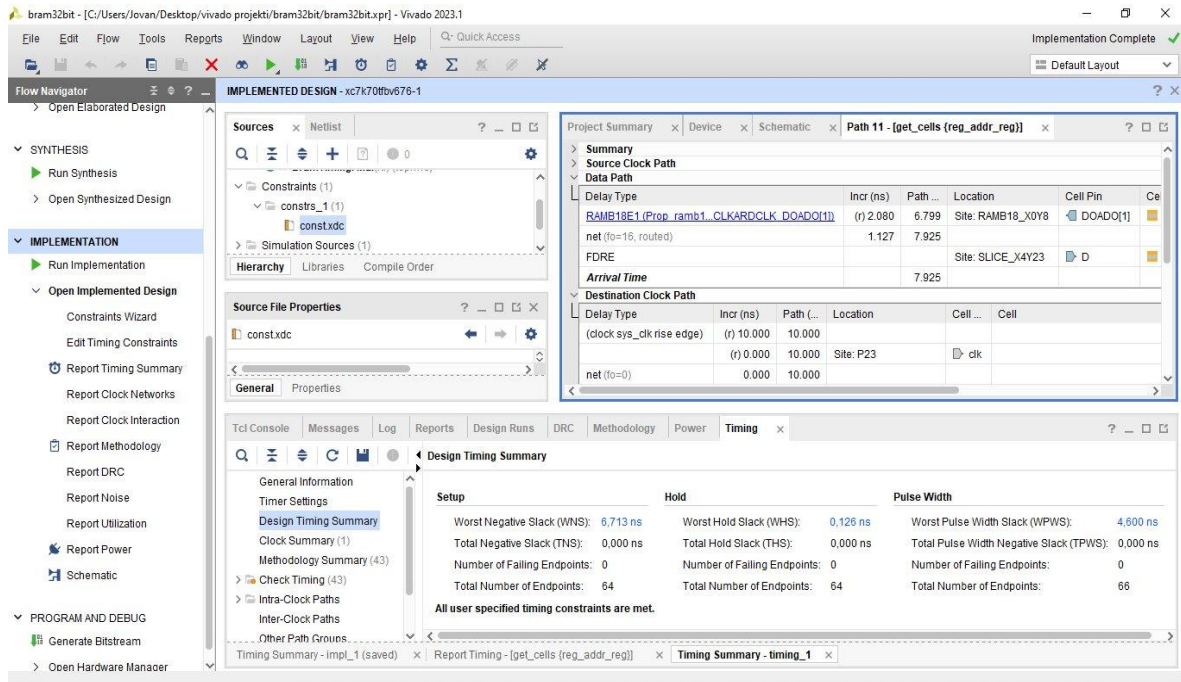
Sa periodom takta od 10 ns, ukupno vreme po jednom pozivu iznosi **0.1036 s**.



Slika 3. Izveštaj sinteze NCC jezgra u alatu Vitis HLS (latencija i utrošeni resursi)

7.2. BRAM kašnjenje

Na osnovu izveštaja iz alata Vivado, kašnjenje **BRAM** modula iznosi 2.08ns.



Slika 4. Vremenska analiza implementiranog dizajna u alatu Vivado (BRAM)

7.3. DMA i DDR kašnjenje

- **DMA:** vreme koje je potrebno samo za setup (≈ 320 ns), prenos bajtova se preklapa sa NCC obradom.
- **DDR:** ne modeluje se kašnjenje (zaseban, brži PS clock domen).

8. Optimizacije i paralelizacija

8.1. Analiza vremena izvršavanja (pre optimizacija)

Kao polazna tačka za optimizaciju iskorišćeno je izmereno simulaciono vreme osnovne arhitekture sa **jednim NCC blokom**. Za testni fajl `board2.txt` (koji sadrži 32 popunjena polja), ukupno vreme izvršavanja iznosi $39.791.629.690 \text{ ns} \approx \mathbf{39.79 \text{ s}}$.

$$39.79 \text{ s} = 32 \text{ polja} \times 12 \text{ templejta} \times 0.1036 \text{ s} = \mathbf{384 \text{ NCC poziva}} \times 0.1036 \text{ s}$$

Pojedinačni NCC poziv traje **0.1036 s**, pa ukupno vreme direktno potiče od 384 uzastopna poziva. S druge strane, kašnjenja nastala usled konfigurisanja DMA kontrolera (~10 us ukupno) i pristupa BRAM memoriji (~8.5 ms ukupno) su zanemarljiva u odnosu na ukupno vreme koje troši NCC.

8.2. HLS resursna analiza i broj NCC blokova

Iz Vitis HLS izveštaja, resursi **jedne** NCC instance i kapaciteti XC7Z010:

Resurs	Po instanci	Kapaciteti XC7Z010	% po instanci	Max instanci (100%)
LUT	5269	17.600	29.9%	$[3.34] = \mathbf{3} \leftarrow$ vezujući
DSP48E	16	80	20.0%	$[5.0] = 5$
FF	3455	35.200	9.8%	$[10.2] = 10$
BRAM_18K	4	120	3.3%	$[30] = 30$

Tabela 6. Resursi ploče i njihova iskorišćenost na osnovu broja instanci IP-ja

Tri instance bi bio teorijski maksimum, ali bi koristile ~90% LUT-ova, što znači da ne ostaje prostora za AXI interkonekt, DMA, BRAM kontroler i rutiranje, te ova opcija nije ostvarljiva. **Dve instance** su bezbedan i realan izbor, zauzele bi $2 \times 5269 = 10.538$ LUT-ova (~**60%**), što ostavlja ~40% za ostatak sistema i ukupno vreme obrade bilo bi **upola manje**.

8.3. Paralelizacija (2 NCC bloka)

Dodat je drugi NCC blok (NCC1 na 0x51000000). Templejti se obrađuju u **parovima**: NCC0 dobija templejt t, NCC1 templejt t+1. Oba bloka rade nad istim segmentom istovremeno, pa se 12 templejta prođe u **6** rundi umesto u 12.

Sinhronizacija u testbench-u koristi SystemC `sc_event_and_list`:

```
if (use_a && use_b)
    // Čekaj signal završetka sa OBA hardverska bloka
    wait(*m_ncc0_done & *m_ncc1_done);
```

Nakon implementacije dva paralelna NCC bloka, izveštaj o zauzeću resursa na čipu **XC7Z010** izgleda ovako:

Resurs	2xNCC	Kapaciteti XC7Z010	Iskorišćenost
LUT	10.538	17.600	59.9%
DSP48E	32	80	40.0%
FF	6.910	35.200	19.6%
BRAM_18K	8	120	6.7%

Tabela 7. Resursi ploče i njihova iskorišćenost nakon dodavanja još jedne instance

8.4. Optimizacije

8.4.1. Provera praznog polja

Kako bi se izbegao nepotreban rad, uvedena je **softverska provera praznih polja** pre poketanja hardverskih akceleratora. CPU direktno iz DDR memorije analizira centralnu zonu polja i poredi njenu srednju vrednost sa pikselom u uglu. Ukoliko se zaključi da je polje prazno (što je slučaj sa preostala 32 polja na tabli koja je korišćena za testiranje), **preskaču se i DMA transfer i svih 12 NCC poziva**, čime se štede procesorski ciklusi.

8.4.2. Keširanje učitavanja segmenta

NCC je ranije čitao ceo segment iz BRAM-a na svakom okidanju (`REG_CTRL = 1`), iako se segment ne menja (menjaju se šabloni). Uveden je flag `img_dirty` koji se postavlja na upis `REG_IMG_ADDR`; slika se čita samo tada. Time se broj čitanja slike smanjuje. Takođe se i integralne sume izračunavaju samo jednom za svaki segment. Svako naredno okidanje obrade za isti segment preskače ponovno čitanje i koristi keširanu sliku i već izgrađenu SAT tabelu.

Invalidacija keša je vezana za upis registra `REG_IMG_ADDR`, a ne za vrednost adrese. To je bezbedno iako se isti BRAM bafer (`BRAM_SEG_FULL = 0x80000`) reciklira za svako novo polje: CPU upisuje adresu segmenta na početku obrade svakog polja, čime se keš pouzdano osvežava i kada se numerička adresa ne promeni.

8.4.3. Pred-klasifikacija boje figura

Šahovski set ima 6 tipova figura $\times$ 2 boje = 12 šablona. Pošto je vreme simulacije proporcionalno broju NCC poziva, ako se boja figure odredi pre korelacije, testira se samo 6 šablona te boje umesto svih 12.

Boja se određuje iz iste centralne srednje vrednosti koja se već računa za detekciju praznog polja (zona 31×31 u centru segmenta). Bele figure imaju visok intenzitet, a crne nizak.

- crne figure: centralna srednja vrednost 81 – 116
- bele figure: centralna srednja vrednost 178 – 227
- prag: 140

```
// tb.cpp
bool is_white = (mean > COLOR_THRESHOLD); // 140
for (int i = 0; i < 6; i++)
    cands.push_back(is_white ? i : 6 + i);
```

Pošto pozadina polja (svetlo/tamno) ne ulazi u centralnu zonu dominantno, prag radi nezavisno od boje polja.

8.4.4. Coarse-to-fine (downsampling 2×)

Cena NCC-a raste sa pozicije ($\text{tmp_w} \cdot \text{tmp_h}$), pa smanjenje rezolucije za faktor k smanjuje rad $\sim k^4$ ($k=2 \rightarrow \sim 16\times$). Ali rezolucija se mora smanjiti i slici i šablonu istim faktorom (NCC poredi na istoj skali)

Rezoluciju smanjujemo softverski u dva nivoa:

- šabloni se smanje jednom pri učitavanju ($30 \times 30 \rightarrow 15 \times 15$) i upišu u BRAM region `BRAM_TMP_COARSE` (`0x10000`),
- segment se po polju smanji $90 \times 90 \rightarrow 45 \times 45$ i upiše u `BRAM_SEG_COARSE` (`0x90000`).

Smanjenje rezolucije se vrši na sledeći način:

```
dst[y*dw+x] = (src[2y*w+2x] + src[2y*w+2x+1]  
               + src[(2y+1)*w+2x] + src[(2y+1)*w+2x+1] + 2) >> 2;
```

Obrada je dvostepena :

- Korak 1 — grubi screen: NCC na 45×45 nad svih 6 kandidata od kojih se biraju **dva** kandidata sa najboljim rezultatom.
- Korak 2 — fina potvrda: pun NCC na 90×90 samo nad ta 2 kandidata.

Time se po polju umesto 6 punih poziva radi 6 grubih + 2 puna.

8.5. Analiza vremena izvršavanja nakon optimizacija

8.5.1. Model latencije zavisen od veličine posla

Da bi se uštede od smanjivanja posla (manji šabloni, grubi screen) uopšte videle u rezultatu, latencija NCC bloka više **nije fiksna**, već je proporcionalna stvarnom poslu — broju pozicija pretrage ($\text{res}_w \cdot \text{res}_h$) pomnoženom veličinom šablona ($\text{tmp}_w \cdot \text{tmp}_h$).

Isti model za grubi poziv (segment 45×45 , šablon 15×15 , $\text{res} = 31 \times 31$):

$$t_{\text{grubi}} = (1000 + 3.094 \times 31^2 \cdot 15^2) \times 10 \text{ ns} \approx 669.910 \times 10 \text{ ns} \approx 0.0067 \text{ s}$$

Grubi poziv tako košta $\sim 16\times$ manje od punog ($k=2 \rightarrow k^4 \approx 16$), čime coarse-to-fine i pred-klasifikacija boje donose stvarnu uštedu simulacionog vremena, a ne samo formalno smanjenje broja operacija.

8.5.2. Dekompozicija vremena po popunjenom polju

Sa svim optimizacijama (2 NCC bloka + boja + coarse-to-fine), obrada jednog popunjenog polja umesto 12 punih poziva izgleda ovako:

Faza	Pozivi	Rundi (po 2 bloka)	Vreme
Korak 1	6 kandidata (jedna boja)	3	20 ms
Korak 2	2 kandidata (top 2)	1	104 ms
Ukupno po polju:			124 ms

Tabela 8. Pregled vremena obrade jednog polja

Za poređenje, polazna obrada je po polju radila 12 punih poziva (sekvencijalno): $12 \times 0.1036 \text{ s} \approx 1243 \text{ ms}$. Uštedu od $\sim 10\times$ po polju donose tri nezavisna faktora koja se množe:

- pred-klasifikacija boje: $12 \rightarrow 6$ kandidata ($2\times$),
- coarse-to-fine: 6 punih poziva $\rightarrow$ 6 grubih (≈ 1 pun) + 2 puna po polju,
- paralelizacija (2 bloka): preostale runde teku po dve istovremeno.

8.5.3. Pregled rezultata

U narednoj tabeli prikazana su simulaciona vremena kroz različite verzije platforme i ubrzanje koje je postignuto nakon optimizacija.

Konfiguracija	Simulaciono vreme	Ukupno ubrzanje
1 NCC (polazno)	39.79 s	1x
2 NCC paralelno (fiksna latencija)	19.89 s	2.0x
2 NCC + optimizacije	3.667 s	10.8x

Tabela 9. Pregled simulacionog vremena i ubrzanja kroz verzije platforme

Paralelizacija sama daje očekivanih $2\times$ ($32 \text{ polja} \times 6 \text{ rundi} \times 0.1036 \text{ s} \approx 19.9 \text{ s}$). Dodavanje pred-klasifikacije boje, smanjenje rezolucije i latencije zavisne od veličine spušta vreme na 3.667 s — dodatnih $5.4\times$ u odnosu na samu paralelizaciju, odnosno $10.8\times$ u odnosu na polaznu arhitekturu.

Idealizovani proračun ($124 \text{ ms/polje} \times 32 \approx 3.96 \text{ s}$) blago precenjuje izmereno vreme (3.667 s) jer stvarni šabloni nisu svi tačno 30×30 (pešaci i drugi su manji), pa su i puni i grubi pozivi u proseku jeftiniji od referentnih.

Najbitnije je da je generisani FEN u svim konfiguracijama identičan:

“rnbqkbnr/pp5p/4ppp1/2pp4/5P2/1P1BPN2/P1PPQ1PP/RNB1K2R”

što potvrđuje da optimizacije ubrzavaju obradu bez gubitka tačnosti.